Intelligent Web Data Aggregator

LLM-assisted Web Extraction API

Student: Yan Marchan

Supervisor: Evgeniy Tretyak

Jan 2026

Motivation

- Teams need fresh web data: **prices**, **job posts**, **news**, **catalogs**, **events**.
- Modern sites are dynamic (JavaScript rendering) and change layout frequently.
- Classic scrapers rely on brittle patterns (`CSS/XPath/regex`) and require constant maintenance.
- Goal: reduce “developer-in-the-loop” parsing work and speed up onboarding new sources.

Goal & Scope

- **Input:** URL + natural language prompt (“extract title and price”).
- **Output:** structured JSON stored in DB and accessible via API.
- **Async by design:** create task → poll status/result.
- **Scope boundaries:** public pages; no CAPTCHA solving; one URL per task (no crawler).
- **Automation:** user-owned cron schedules for recurring collection.

Key Contributions

- **Semantic content extraction:** enrich visible text with role markers (`[LINK]`, `[BUTTON]`, headings, tables, images).
- **Dual extraction paths:** schema (multi-field) and single-field pipelines.
- **LLM-assisted parser generation:** build parsers (regex/CSS/JSONPath) with validation and retries.
- **Parser cache + self-healing:** reuse on cache-hit; auto-invalidate when stale.
- **Production basics:** JWT auth, rate limiting, prompt-injection sanitization, health checks, correlation IDs.

System Architecture

```
Client
  |
  v
FastAPI API (port 8000)  ----->  PostgreSQL 15 (tasks, results, cache)
  |
  v
Redis (Celery broker / rate-limit store)
  |
  +--> AI Worker (queue: ai_queue)  <----- Scheduler (Celery Beat)
  |
  +--> Headless Worker (queue: fetching_queue, Playwright) ---> Target Websites
```

- **Stack:** FastAPI, Celery, Redis, Postgres, Playwright, LiteLLM (Baseten/DeepSeek, Gemini, optional OpenAI), Docker Compose.

End-to-End Flow

1. **Create task:** `POST /api/v1/process` (JWT required).
2. **Intent extraction:** AI Worker converts prompt → `keywords` / `schema_fields`.
3. **Fetch:** Headless Worker renders page with Playwright and captures semantic text + HTML.
4. **Cache-first:** try validated parsers for the same URL + fields.
5. **If cache miss:** generate a parser (regex/CSS/JSONPath) and validate it before caching.
6. **Persist result:** save JSON + metadata; client polls `/api/v1/status` and `/api/v1/result`.

API Interface

- **Docs:** Swagger UI at `/docs`
- **Auth:** `POST /auth/register`, `POST /auth/token` → Bearer token
- **Tasks:** `POST /api/v1/process`, `GET /api/v1/status/{task_id}`, `GET /api/v1/result/{task_id}`
- **Scheduling:** `POST/GET/DELETE /api/v1/jobs`
- **Rate limits:** process 10/min, register 5/min, login 10/min (per IP)

```
POST /api/v1/process
{
  "url": "https://example.com/page",
  "prompt": "Extract product title and price"
}
```

Headless Worker (Playwright)

- **Purpose:** render JavaScript-heavy pages and capture extraction-friendly signals.
- **Captured artifacts:**
  - `inner_text`: visible body text
  - `semantic_text`: enriched text with markers (`[LINK]`, `[BUTTON]`, headings, tables, image alt/URLs)
  - `html`: full DOM snapshot (stored with defensive cap)
  - `network`: selected XHR/fetch/document/script events (preview)
- **Stealth basics:** randomized UA/viewport, init script, cookie banner dismissal attempts.

Intent Extraction & Safety

- **Prompt-injection defense:** sanitize user input before queueing work (defense in depth).
- **Intent extraction:** LLM converts prompt into structured intent:
  - `keywords` (single-field extraction)
  - `schema_fields` (multi-field structured extraction)
  - `target` (what the user wants overall)
- **Normalization:** field names are normalized for consistency (`"Price"` → `price`).
- **Traceability:** intent is stored with the task to explain “why this result”.

Extraction Pipeline

- **Cache-first:** reuse validated parsers for the same URL + fields.
- **Two paths:**
  - **Schema extraction:** LLM returns structured items with field associations.
  - **Single-field extraction:** find examples → locate snippets → generate parser → validate.
- **Parser types:** regex (common), CSS selector, JSONPath (when applicable).
- **Quality gate:** generate → execute → validate → retry; only validated parsers are cached.
- **Token optimization:** regex generation uses focused snippets (e.g., ≤32k chars), not full HTML.

Parser Cache & Self-Healing

- **Cache key:** domain + **full URL** + fields (+ source type).
- **Stored:** parser pattern (regex/CSS/JSONPath), confidence, samples, and usage metadata.
- **Auto-invalidation:** if a cached parser stops working (0/invalid matches), it is removed and regenerated.
- **Benefit:** fewer LLM calls on repeated pages and faster extraction on cache hits.

Cache Hit Check (Same URL + Intent)

- **Intent normalization:** prompt → intent → normalized fields (`schema_fields` / `keywords`).
- **Lookup key:** `(domain, full_url, fields, source_type)` (exact URL match).
- **First run:** Headless Worker always captures `semantic_text` and stores it as task `page_content` for traceability.
- **Cache hit:** apply cached parser locally → skip LLM generation → `used_cached_parser=true`.
- **Cache miss:** run LLM extraction → generate + validate parser → cache it for next time.

```
fields = normalize(intent.schema_fields or intent.keywords)
key    = (domain, full_url, fields)

if cache.has(key, source="SEMANTIC"):
    data = run_parser(cache[key], semantic_text)
```

Scheduling

- **Feature:** recurring collection for the same URL + prompt.
- **API:** create cron schedule via `POST /api/v1/jobs`.
- **Implementation:** Scheduler service runs Celery Beat and enqueues due jobs every minute.
- **Result:** each run creates a normal scraping task with stored history/results.

Persistence Model (PostgreSQL)

Entity	Purpose
Users	JWT authentication + ownership of tasks and schedules
ScrapingTasks	Status lifecycle, prompt, intent (JSON), sources (text/HTML), extracted JSON results
ScheduledJobs	Cron schedules linked to user; next-run bookkeeping
ParserCache	Validated parsers (regex/CSS/JSONPath) for reuse + samples/confidence

Reliability & Observability

- **Correlation IDs:** `X-Correlation-Id` propagated API → Celery → workers for traceable logs.
- **Health:** `/health` (liveness) and `/api/v1/health` (includes DB check).
- **Timeouts/retries:** Playwright + LLM timeouts configurable via environment variables.
- **Dockerized:** services have health checks and compose dependency conditions.

Security Controls

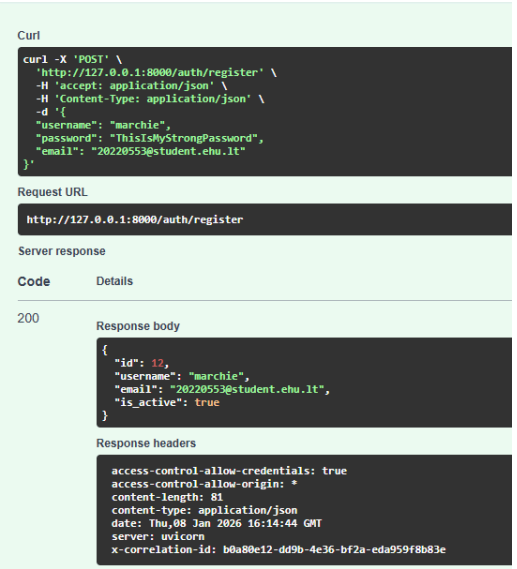
- **JWT auth:** all task and scheduler endpoints require a Bearer token.
- **Rate limiting:** slowapi limits key endpoints to reduce abuse.
- **Input sanitization:** blocks prompt-injection/jailbreak patterns before processing.
- **Secrets:** API keys and credentials are provided via environment variables (no secrets in code).

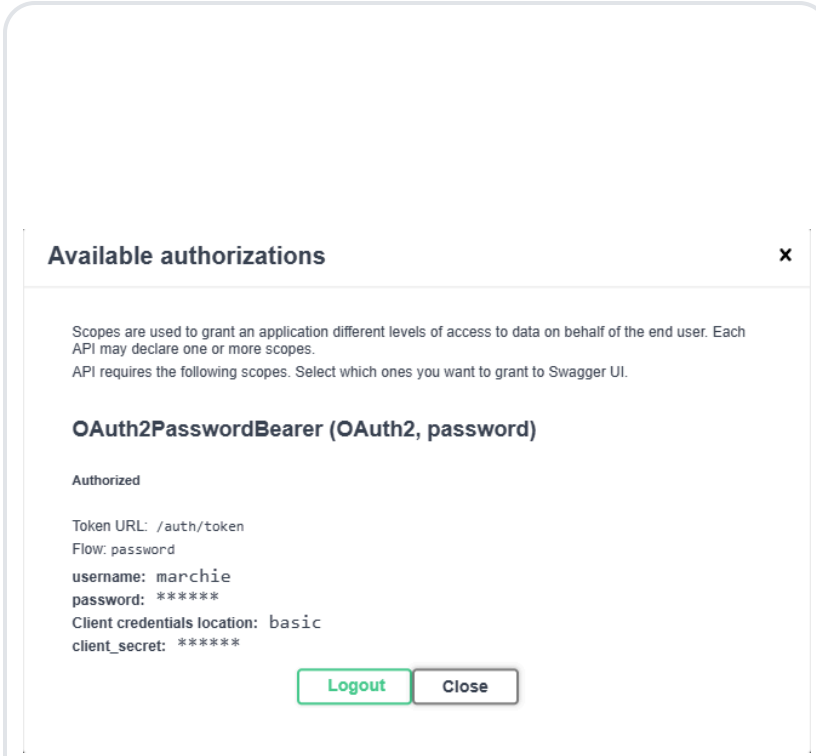
Testing & Quality

- **Pytest suite:** unit + integration + contract/snapshot + workflow tests.
- **API contract:** OpenAPI endpoints documented and tested for stability.
- **Coverage:** overall **78%** (see `coverage_report.txt`).

```
Coverage summary (pytest-cov):
TOTAL  4168 statements, 926 missed  →  78% covered
```

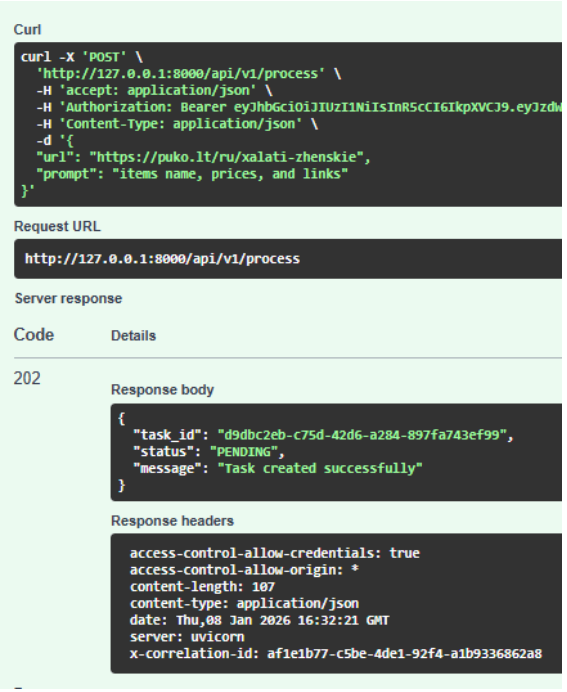
Demo Screenshots (1/3) — Authentication

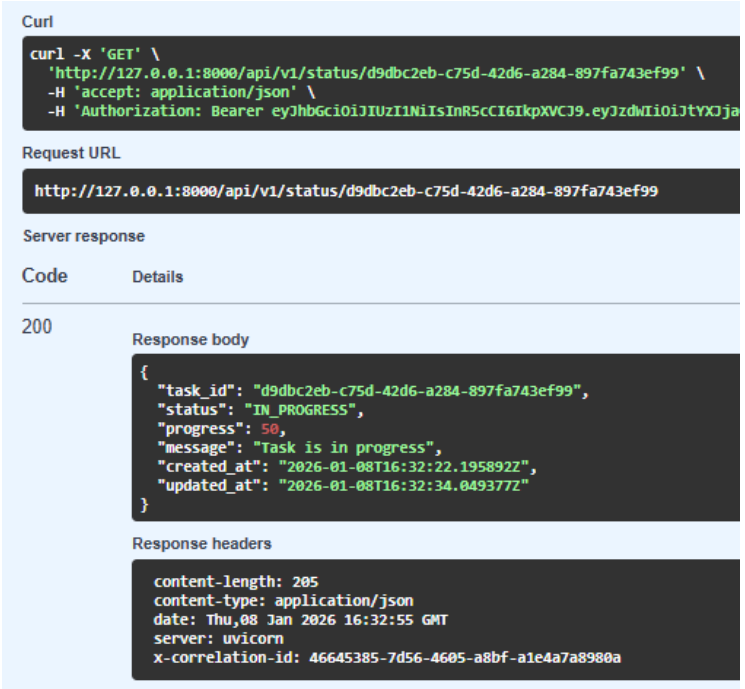




- **Flow:** register → login → get JWT token
- **Show:** Swagger “Authorize” with Bearer token
- **Optional:** invalid login error state

Demo Screenshots (2/3) — Create Task & Poll Status





- **Input:** URL + natural language prompt
- **Response:** 202 Accepted + `task_id`
- **Status gathering:** poll `/api/v1/status/{task_id}` until `SUCCESS`

Demo Screenshots (3/3) — Fetch Result

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/api/v1/result/d9dbc2eb-c75d-42d6-a284-897fa743ef99' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJtYXJjaGllIiwiZXhwIjoxNzY3ODkxMDM0fQ.1iAJ_Q64POJmHLAgU_F
```

Request URL

```
http://127.0.0.1:8000/api/v1/result/d9dbc2eb-c75d-42d6-a284-897fa743ef99
```

Server response

Code	Details
200	Response body


```
{
  "task_id": "d9dbc2eb-c75d-42d6-a284-897fa743ef99",
  "status": "SUCCESS",
  "url": "https://puko.lt/ru/xalati-zhenskie",
  "prompt": "items name, prices, and links",
  "data": [
    {
      "text": "name: Халат Бамбоо весна | price: 75.00€ | link: https://puko.lt/ru/xalaty-dlinyje-kapishon-vesna",
      "fields": {
        "name": "Халат Бамбоо весна",
        "price": "75.00€",
        "link": "https://puko.lt/ru/xalaty-dlinyje-kapishon-vesna"
      },
      "source": "schema_extraction",
      "confidence": 0.9
    },
    {
      "text": "name: Халат Бамбоо Копи | price: 54.00€ | link: https://puko.lt/ru/xalaty- kapishon-bambuk-copi",
      "fields": {
        "name": "Халат Бамбоо Копи",
        "price": "54.00€",
        "link": "https://puko.lt/ru/xalaty- kapishon-bambuk-copi"
      },
      "source": "schema_extraction",
      "confidence": 0.9
    },
    {
```


- **When ready:** `GET /api/v1/result/{task_id}`
- **Data:** structured items with `fields` (e.g., name/price/link)
- **Metadata:** `total_matches` + `used_cached_parser`

Thank You

Q & A

Docs: `docs_old/ARCHITECTURE.md`, `docs_old/USER_GUIDE.md`

API: `http://localhost:8000/docs`